

SentinelShark

A Cross-Platform Network Intrusion Detection and Threat-Correlated Packet Analysis System

Project Synopsis

Submitted in partial fulfillment of the requirements for the
Bachelor of Technology in Computer Science and Engineering

Submitted by:

Debdoot Manna

Final Year, B.Tech Computer Science

CSPIT, Charusat University

Internship Host Organization:

L&T Carbonlite Solutions, Vadodara

Digital Forensics Internship

Repository:

<https://github.com/DebdootManna/SentinelShark>

August 2026

Abstract

Conventional packet analyzers such as Wireshark and tcpdump excel at raw traffic capture and protocol dissection, but they offer no automated correlation with external threat-intelligence sources. A security analyst must manually copy every observed public IP address into a browser tab and cross-check it against reputation databases – a workflow that does not scale under high packet volumes or in incident-response scenarios where triage speed is critical.

SentinelShark is a desktop Network Intrusion Detection and Analysis System (NIDS) that closes this gap. It combines live packet capture (via TShark, with a PyShark and a synthetic-traffic fallback), full L2–L7 protocol dissection, payload hashing (MD5/SHA256), and an asynchronous threat-intelligence pipeline that queries four external providers – AbuseIPDB, Virus-Total, IPinfo, and Shodan/InternetDB – to compute a composite threat score for every flow. The system presents this information through a dark-themed, Figma-inspired PyQt6 interface with a live packet table, statistics dashboard, protocol dissection tree, hex/ASCII viewer, and bandwidth sparklines.

The project began as a personal, self-directed effort and was subsequently adopted as an internship project at L&T Carbonlite Solutions, where a dedicated high-specification Windows 11 development machine was provided to continue the work. A native C#/.NET 8 (WPF) companion application was built alongside the Python version specifically to produce a portable, dependency-free Windows `.exe`. In practice, the WPF port has proven inferior in both visual polish and development velocity compared to the original PyQt6 application. This synopsis therefore also documents the proposed next phase of the project: rather than continuing to maintain a separate C#/WPF codebase, the Python application itself will be compiled into an optimized, portable native executable using a Python + C++ (or Python + Rust) toolchain, retiring the WPF companion while retaining a single, higher-quality codebase.

1 Introduction

Network environments generate high-velocity, high-volume traffic, and diagnosing compromise, data exfiltration, or denial-of-service activity within that traffic is a persistent bottleneck for security teams. Existing open-source packet analyzers focus almost exclusively on capture and dissection; they treat every IP address as equally unremarkable and leave reputation lookup, risk scoring, and visual triage entirely to the analyst.

SentinelShark was conceived to address this gap directly: to take the raw packet stream that a tool like Wireshark already exposes and enrich it, in real time and without manual effort, with threat context drawn from multiple independent intelligence sources. The result is a tool that not only shows *what* traffic looks like, but also flags *which* flows deserve an analyst's attention first.

The project is being developed and extended as part of a Digital Forensics internship at L&T

Carbonlite Solutions, Vadodara, where it was formally adopted for continued development on office-provided infrastructure after an initial working prototype was demonstrated.

2 Current Project Status (as of August 31, 2026)

The project has moved through prototype, feature-complete, and internship-adoption stages. Table 1 summarizes where each major work-stream currently stands.

Table 1: Current status of major SentinelShark work-streams.

Work-stream	Status	Notes / Recent Update
Live capture (TShark → PyShark → Mock fallback)	Done	Sustains >500 pkts/s; 3-tier fallback verified on macOS and Windows.
Protocol dissection (L2–L7)	Done	DNS, HTTP, HTTPS/TLS, SSH, TCP, UDP, ICMP, ARP, QUIC all decoded.
Threat-intelligence pipeline (4 providers)	Done	AbuseIPDB, VirusTotal, IPinfo, Shodan/InternetDB integrated; TTL cache + backoff in place.
PCAP import / export	Done	Standard libpcap binary read/write, Wireshark-compatible.
PyQt6 desktop UI (Python build)	Done	Dark cyber theme, packet table, dissection tree, hex viewer, sparklines all functional.
Automated test suite	Done	13/13 unit tests passing (caching, hashing, BPF sanitization, dissection).
C# / .NET 8 WPF companion app	Complete, deprioritized	Functionally complete and produces a portable .exe, but UI polish and dev velocity are noticeably behind the Python build.
Internship adoption & infrastructure	Done	L&T Carbonlite Solutions approved continued development; dedicated high-spec Windows 11 machine provisioned.
Dev-machine toolchain setup	Done	Git, GitHub CLI, Wireshark, Python, C++, Rust, and .NET installed on the office Windows 11 machine.

Work-stream	Status	Notes / Recent Update
Python-native portable executable (C++/Rust)	Planned	Not yet started; scheduled as the primary focus of the next development phase (Section 8).
WPF companion retirement	Planned	Contingent on the native Python build reaching parity with the current WPF .exe.

3 Objectives

- O1.** Implement high-throughput live packet capture via direct TShark JSON streaming, with graceful multi-tier fallback (TShark → PyShark → synthetic mock traffic) so the tool remains usable even without elevated privileges or a capture driver.
- O2.** Perform structured protocol dissection across OSI layers L2 through L7 (Ethernet, IPv4/IPv6, TCP/UDP, DNS, HTTP, HTTPS/TLS, SSH, ICMP, ARP, QUIC) and compute per-packet MD5/SHA256 payload hashes.
- O3.** Build an asynchronous threat-intelligence pipeline that queries AbuseIPDB, VirusTotal, IPinfo, and Shodan (with a free InternetDB fallback) without blocking the UI, while respecting API rate limits through caching, a token-bucket limiter, and exponential backoff.
- O4.** Compute a composite, explainable threat score per flow and present it through color-coded visual triage (Safe / Suspicious / Warning / Critical).
- O5.** Support standard Wireshark-compatible PCAP/PCAPNG import and export so SentinelShark can be used alongside existing forensic tooling.
- O6.** Deliver an ergonomic, dark-mode desktop interface (packet table, statistics dashboard, dissection tree, hex viewer, sparkline telemetry) that is usable directly by SOC analysts and incident responders.
- O7.** Ensure full cross-platform behaviour on macOS and Windows, and evaluate distribution as a self-contained, portable executable so the tool can be handed to a non-technical user without a Python environment.
- O8.** (Ongoing) Identify and adopt the most maintainable and performant packaging strategy for a portable Windows executable, replacing the current dual-codebase (Python + C#/WPF) approach with a single, higher-quality Python-based build.

4 Literature Review / Existing Systems

- **Wireshark / tshark** – the de facto standard for packet capture and protocol dissection. Extremely thorough at the protocol level, but offers no built-in threat-reputation correlation; an analyst must manually look up every IP of interest.
- **Snort / Suricata** – signature- and rule-based network intrusion detection engines, well suited to automated, unattended detection at the network perimeter, but they are not interactive packet inspection tools and do not present a live, explorable GUI comparable to Wireshark.
- **Zeek (formerly Bro)** – generates rich network metadata logs for later analysis, but again is not a live, analyst-facing desktop application, and threat-intelligence correlation must be wired in separately.
- **Manual reputation lookup workflows** – the common real-world practice of pasting suspicious IPs one at a time into AbuseIPDB, VirusTotal, or Shodan web pages. This does not scale, is error-prone, and provides no persistent, session-wide view of overall network risk.

SentinelShark’s contribution is not a new detection algorithm but an **integration and workflow improvement**: it unifies capture, dissection, and multi-source threat correlation inside a single interactive desktop application, removing the manual reputation-lookup step that every existing tool leaves to the analyst.

5 Proposed Methodology

5.1 System Architecture

The application follows a strict three-tier layered architecture:

- **UI Presentation Layer** – PyQt6 widgets (`MainWindow`, `PacketTable`, `PacketDetailView`, `HexView`, `StatsPanel`), communicating with lower layers exclusively via Qt signals to keep the interface decoupled from capture and network logic.
- **Core Processing Layer** – capture, dissection, PCAP I/O, caching, and configuration (`capture.py`, `parser.py`, `pcapwriter.py`, `cache.py`, `config.py`).
- **Asynchronous Service Layer** – the threat-intelligence HTTP client and queue manager (`threatintel.py`, `queuemanager.py`), built on `httpx` and `qasync` so that network I/O runs as non-blocking coroutines inside the same event loop as the Qt UI.

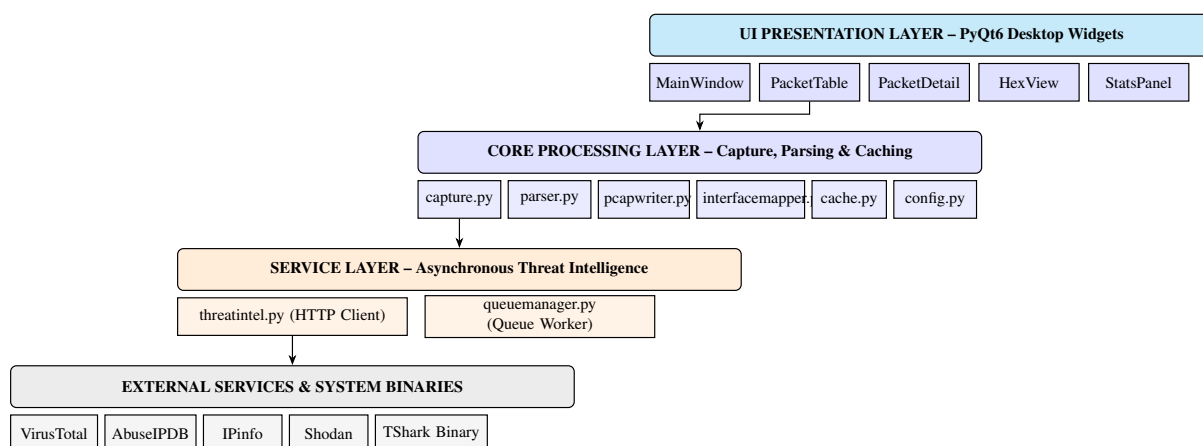


Figure 1: SentinelShark three-tier layered system architecture.

5.2 Concurrency Model

Live capture executes on a dedicated `QThread` (`LiveCaptureThread`) so the ingestion loop never blocks the GUI. Threat-intelligence lookups run as `asyncio` coroutines managed by `qasync`, embedded directly inside the PyQt6 event loop. All inter-thread communication uses `pyqtSignal` objects to guarantee thread safety. Figure 2 shows the multi-tier capture fallback decision flow.

5.3 Threat Correlation

Every observed public (non-RFC 1918, non-loopback, non-multicast) IP address is enqueued for lookup against AbuseIPDB, VirusTotal, IPinfo, and Shodan (or its free InternetDB fallback when no paid key is configured). Results are cached in memory with a 24-hour TTL and LRU eviction, and a composite score is derived as the maximum of the AbuseIPDB confidence score and the VirusTotal malicious-engine count scaled by a fixed factor, with an automatic uplift when Shodan reports known CVEs against the host. The score drives the row-level colour coding in the packet table. Figure 3 details the full enrichment pipeline from packet dissection to cached, colour-coded result.

5.4 Distribution Strategy – Current State and Proposed Revision

The original distribution plan used two parallel codebases:

- The primary **Python 3.10+ / PyQt6** application, used for all local feature development, because Python offers the fastest iteration loop for the capture, parsing, and UI logic.
- A native **C# / .NET 8 (WPF)** companion application, built solely to produce a self-contained, single-file, portable `.exe` for Windows deployment without requiring end users to install a Python runtime.

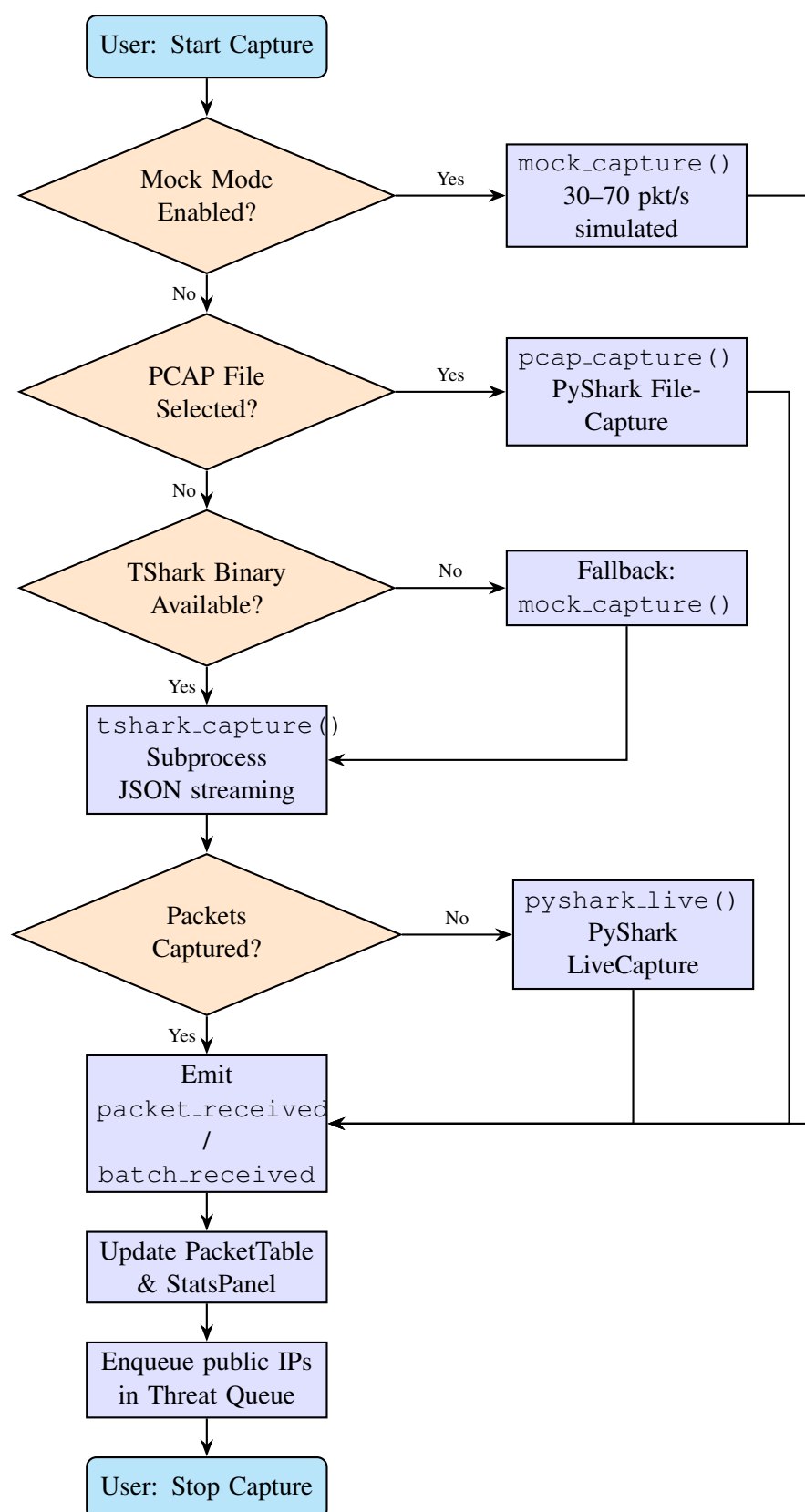


Figure 2: Packet capture engine execution lifecycle and multi-tier fallback.

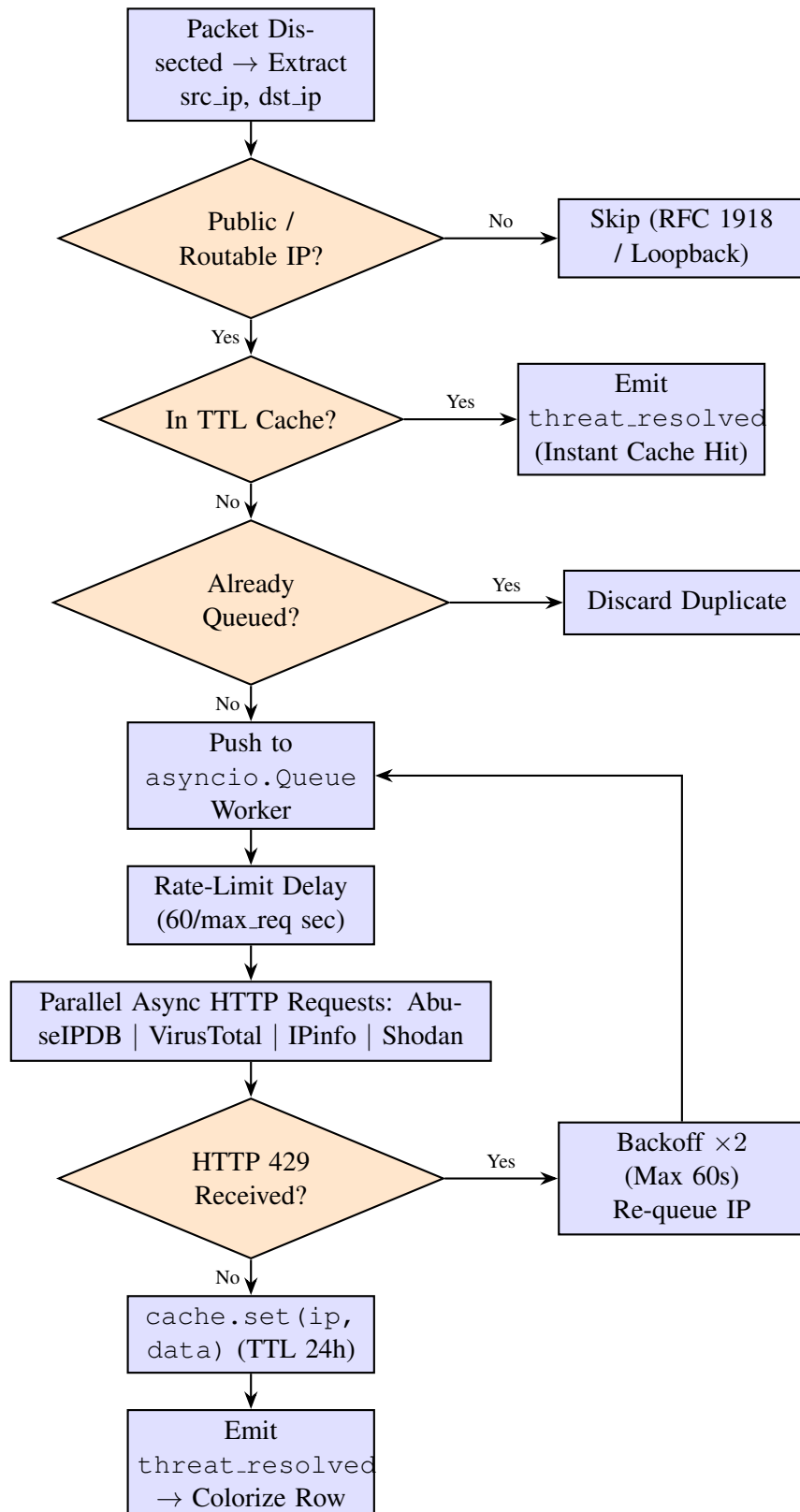


Figure 3: Threat intelligence enrichment pipeline.

In practice, maintaining the WPF port as a full re-implementation of the PyQt6 UI has proven inefficient: development velocity is slower, and the resulting interface is visibly less polished than the original Python/PyQt6 design. It has since become clear that a portable, dependency-free Windows executable can also be produced directly from the Python codebase using a Python + C++ (or Python + Rust) toolchain – for example, packaging with a tool such as PyInstaller or Nuitka, and optionally moving performance-sensitive parsing paths into a compiled C++/Rust extension for speed.

The proposed revision to the methodology is therefore to:

1. Retain the Python/PyQt6 application as the single source of truth for all UI and feature development.
2. Evaluate C++ and Rust as extension/compilation targets for producing an optimized, portable native executable directly from the Python codebase, benchmarking both against the current WPF build for startup time, packet-processing throughput, and binary size.
3. Retire the separate C#/WPF companion once the Python-based portable build reaches parity or better on Windows, eliminating the duplicate-codebase maintenance burden.

6 Feasibility Study

6.1 Technical Feasibility

A complete, working prototype already exists and has been demonstrated: live and mock capture, full protocol dissection, PCAP import/export, four-provider threat-intelligence correlation, and an automated 13-case unit test suite are all functional in the current Python codebase (approximately 4,800 lines) and its WPF companion (approximately 2,800 lines). The core technical risk – whether real-time capture and asynchronous threat lookups can coexist without blocking the UI – has already been resolved via the `QThread/ qasync` concurrency model.

6.2 Operational Feasibility

The internship host organization, L&T Carbonlite Solutions, has agreed to continue development of this project and has provided a dedicated, high-specification Windows 11 machine for further work. That machine has been set up with Git, GitHub CLI, Wireshark, Python, and both C++ and Rust toolchains alongside .NET, so that the proposed C++/Rust-based packaging approach can be evaluated without additional procurement.

6.3 Economic Feasibility

All core dependencies – PyQt6, PyShark, httpx, cachetools, psutil, TShark, and the .NET 8 SDK – are free and open source. The threat-intelligence providers used (AbuseIPDB, Virus-

Total, IPinfo, and Shodan InternetDB) all offer functional free tiers sufficient for development and demonstration use, keeping the project’s operating cost effectively nil.

6.4 Schedule Feasibility

The Python application and its feature set are already complete and tested. Remaining work – evaluating and switching the portable-executable build pipeline – is a packaging and optimization task rather than new feature development, and fits comfortably within the remaining internship timeline.

7 Tools, Hardware, and Software Requirements

7.1 Development Hardware

- High-specification Windows 11 desktop/laptop (provided by L&T Carbonlite Solutions) for Windows-side development, WPF and native-executable builds, and cross-platform testing.
- Personal development machine (macOS) for the primary Python/PyQt6 development workflow.

7.2 Software and Toolchain

Category		Tools / Technologies
Core language (primary)		Python 3.10+
GUI framework		PyQt6, custom QSS dark theme
Async runtime		qasync, asyncio, httpx
Packet capture/parsing		TShark, PyShark
Caching		cachetools (in-memory TTL cache)
System telemetry		psutil
Companion executable (current)		C# / .NET 8, WPF
Proposed executable pipeline		Python + C++ and/or Python + Rust (evaluated against PyInstaller/Nuitka-based builds)
Version control & col-laboration		Git, GitHub, GitHub CLI

Category	Tools / Technologies
Design prototyping	Figma, React 19 / Tailwind CSS v4 (UI mockup)
Threat-intelligence APIs	AbuseIPDB v2, VirusTotal v3, IPinfo, Shodan / InternetDB
Testing	Python <code>unittest</code> (13-case automated suite)
Target operating systems	macOS 12+ (Darwin), Windows 10/11 (x64)

Note: Java was also installed on the internship machine as part of general environment setup and is not currently used by any SentinelShark component.

8 Future Development Roadmap (Aug 31 – Oct 31, 2026)

With the Python/PyQt6 application feature-complete, the remaining internship timeline is dedicated to resolving the distribution question raised in Section 5.4: replacing the C#/WPF companion with a native, optimized portable executable built directly from the Python codebase. Figure 4 lays out this plan across nine working weeks, starting today (August 31, 2026) and ending October 31, 2026.

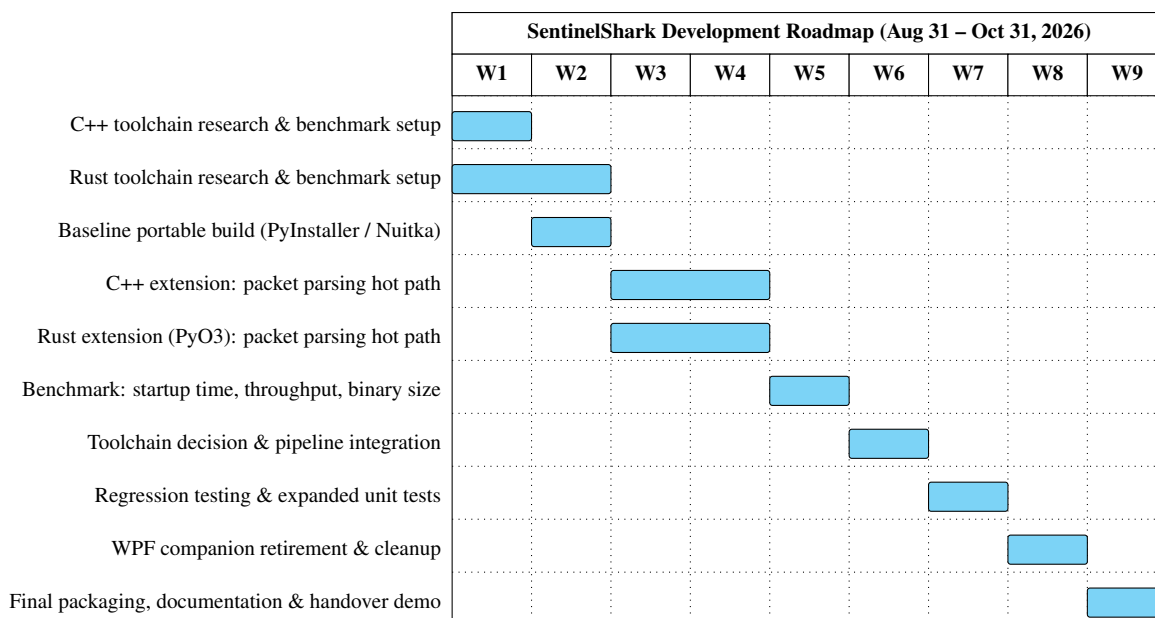


Figure 4: Proposed development timeline from today (August 31, 2026) through October 31, 2026.

Week	Date Range	Week	Date Range
W1	Aug 31 – Sep 6	W6	Oct 5 – Oct 11
W2	Sep 7 – Sep 13	W7	Oct 12 – Oct 18
W3	Sep 14 – Sep 20	W8	Oct 19 – Oct 25
W4	Sep 21 – Sep 27	W9	Oct 26 – Oct 31
W5	Sep 28 – Oct 4		

9 Expected Outcomes

- A production-ready, cross-platform desktop NIDS that gives security analysts an at-a-glance, colour-coded view of network risk without manual IP lookups, cutting incident-triage time relative to a plain Wireshark workflow.
- A single-source, portable Windows executable produced directly from the Python/PyQt6 codebase, matching or exceeding the visual quality of the current UI while removing the need to separately maintain a C#/WPF port.
- A measurable reduction in redundant threat-intelligence API calls (via TTL caching) and in UI repaint overhead (via batched signal emission), keeping the tool responsive above 500 packets/second.
- A documented, tested (13 automated unit tests) and internship- validated project suitable as both a portfolio piece and a candidate for continued internal use at L&T Carbonlite Solutions.
- A clear, benchmarked recommendation on whether C++ or Rust is the better long-term compiled-extension/packaging partner for the Python codebase, closing out the open packaging question raised during this phase of the internship.